UE5 多人連線基礎概念 (Multiplayer Basics)

1. 多人連線的基本型態

本機多人連線 (Local Multiplayer)： 所有玩家在同一台電腦上遊玩（如分割畫面），單一遊戲實體，不需要網路傳輸。

真正多人連線 (True Multiplayer)： 兩個以上的遊戲實體 (Instance) 運行在不同的實體機器上，必須透過網路連輸來同步遊戲資訊。

架構類型	運作原理	優點	缺點
點對點 (Peer-to-Peer, P2P)	玩家之間直接互相傳輸資料。	實作簡單，適合輕量雙人遊戲。	玩家變多時網路流量暴增（擴展性差）；沒有「權威版本」，容易產生資料不同步。
用戶端-伺服器 (Client-Server)	指定一台為「伺服器」，其餘皆為「客戶端」。客戶端彼此不直接溝通，只與伺服器溝通。	大幅減少頻寬消耗；擁有絕對正確的「權威版本」。	實作邏輯較複雜，存在網路傳輸延遲（需要延遲補償）。

UE5 引擎設定： Unreal Engine 原生不支援 P2P 架構，開箱即用的是「具權威性的用戶端-伺服器架構」。

用戶端-伺服器架構：兩種伺服器類型

伺服器類型	說明	適用場景	優缺點
傾聽伺服器 (Listen Server)	由其中一位玩家（房主/Host）兼任伺服器，同時渲染遊戲畫面並處理伺服	休閒遊戲、玩家人數少、合作通關遊戲。	**優點：** 成本低（免租伺服器）。 **缺點：** 房主有零延遲優勢（Host Advantage）；加重房主電腦效能負擔。

伺服器類型	說明	適用場景	優缺點
	器邏輯。		
專用伺服器 (Dedicated Server)	獨立機器純粹處理伺服器邏輯，不渲染畫面，沒有真人在上面遊玩。	競技型射擊遊戲、MMO、電競賽事、高玩家數遊戲。	**優點：** 絕對公平無房主優勢；效能專注於運算。 **缺點：** 維護與硬體/雲端代管成本較高。

4. 核心連線術語 (Core Concepts)
權威性 (Authority)： 伺服器運行的遊戲版本被視為唯一「正確」的版本。客戶端想做任何事（如移動）都是向伺服器發送「請求」，由伺服器驗證合法性後才執行。

網路同步 (Replication)： 伺服器驗證資料後，將更新的資訊「向下發布」並同步給所有客戶端的過程。

延遲補償 (Lag Compensation)： 為了解決輸入請求到伺服器來回的時間差，系統會先讓客戶端畫面立即反應（如角色先移動），再由伺服器事後驗證。若無效，伺服器會強制拉回糾正。

引擎小工具： 在 UE5 中，上述提到的「延遲補償」與「移動預測」機制，已經完美內建於 Character Movement Component (角色移動元件) 中，開發者不需從頭造輪子！

UE5 Actor 網路同步基礎 (Actor Replication Basics)
1. 核心同步屬性 (Core Concepts)
在多人連線中，Actor 能否在伺服器與客戶端之間共享資訊，取決於以下三個核心屬性：

Net Load on Client (在客戶端載入)： * 決定直接放置在地圖中的 Actor，是否要在客戶端載入。若設為 false，該物件只會存在於伺服器端。

Replicates (是否同步)： * 設為 true 會將此 Actor 標記為「可同步」，允許它本身及其變數進行網路同步。

注意：勾選此項會強制讓 Net Load on Client 也生效。

Replicate Movement (同步移動)： * Replicates 只允許變數同步，不包含位置移動。若希望 Actor 的移動（位置資料）也能同步，必須額外開啟此選項。

2. C++ 語法與實作 (C++ Implementation)

在 C++ 建構子 (Constructor) 中設定這三個屬性時，有特定的語法和必須遵守的規則：

```cpp
AMP_Actor::AMP_Actor()
{
    // 1. 設定客戶端載入
    bNetLoadOnClient = true;

    // 2. 設定 Actor 同步
    // ❌ 錯誤寫法：在建構子中呼叫 SetReplicates(true) 會報錯，因為 Actor 尚未初始化。
    // ✅ 正確寫法：直接設定變數
    bReplicates = true;

    // 3. 設定移動同步
    // ❌ 錯誤寫法：SetReplicateMovement(true) 是虛擬函式，不建議在建構子內呼叫。
    // ✅ 正確寫法：使用 UE 提供的非虛擬函式
    SetReplicatingMovement(true);
}
```

3. 注意事項與避坑指南 (Tips & Warnings)
   橡皮筋效應 (Rubber banding) 的產生原因：

情形 A (碰撞不同步)： 伺服器有實體物件，但客戶端沒載入 (Net Load on Client = false)。客戶端走過去時，伺服器判定撞牆，會強行把客戶端的位置拉回原處。

情形 B (位置不同步)： 開啟了物理模擬卻沒開 Replicate Movement。雙方看到的物件位置不同，客戶端跳上物件時，伺服器判定該處無物件而強行拉回玩家。

物理模擬的「致命陷阱」 (Root Component Rule)：

規則： 如果一個 Actor 啟用了移動同步與物理模擬，該物理網格體 (Static Mesh) 必須是 Actor 的「根元件 (Root Component)」。

原因： Replicate Movement 永遠只針對「根元件」的座標進行同步。如果 Static Mesh 只是附屬於預設的 Scene Component 之下，物理碰撞只會讓網格體飛出去，根元件依然留在原地。這會導致伺服器與客戶端的位置徹底脫軌，引發嚴重的橡皮筋效應！

UE5 網路權威與角色 (Authority & Net Role)

1. 核心概念：權威 (Authority)

「權威」代表著**哪一台機器（伺服器或客戶端）負責掌控這個 Actor**。

伺服器權威 (Server Authority)： 絕大多數需要網路同步的 Actor 都是由伺服器生成 (Spawn) 的，因此伺服器對它們擁有絕對的權威。

客戶端權威 (Client Authority)： 如果一個 Actor 是由「客戶端」自行生成的，那該客戶端就擁有它的權威。但注意，網路同步是「單向」從伺服器傳給客戶端的，所以客戶端生成的 Actor **無法**同步給伺服器或其他玩家。

2. 核心概念：本地網路角色 (Local Net Role)

每個 Actor 在不同機器上（伺服器端 vs. 客戶端）會有不同的「身份」，這在 UE5 中被稱為 ENetRole。

網路角色 (ENetRole)	意義與情境說明	舉例
Authority (權威)	掌控該 Actor 的機器。對於同步物件，通常就是指「伺服器端」。	伺服器上的所有 Pawn、Character 與同步 Actor。
Autonomous Proxy (自主代理)	玩家**本機正在控制**的那個角色。擁有預測移動與發送輸入請求的能力。	你電腦螢幕上，受你鍵盤/搖桿控制的那個主角。
Simulated Proxy (模擬代理)	**由伺服器掌控、接收更新資訊**的其他 Actor。	你電腦螢幕上看到的「其他玩家」或「會動的同步方塊」。

重點結論： 在客戶端機器上，只有「你自己控制的角色」是 Autonomous Proxy，其他所有玩家和同步物件都是 Simulated Proxy。

3. 遠端網路角色 (Remote Role)

遠端角色可以理解為**「另一端的視角」**，通常是本地角色的反相狀態

在伺服器上檢查 (Local = Authority)： 它的遠端角色 (Remote Role) 會是 `Simulated Proxy`（代表它在客戶端上的身份）。

在客戶端上檢查 (Local = Proxy)： 它的遠端角色 (Remote Role) 會是 `Authority`（代表它在伺服器端擁有權威）

4. C++ 語法與實作 (C++ Implementation)

在 C++ 中，我們經常使用條件判斷來確保特定邏輯只在對應的機器上執行。

- **檢查是否擁有權威：**

C++

```
if (HasAuthority()) {
    // 這裡的程式碼只會在擁有權威的機器（通常是 Server）上執行
}
```

- **取得本地網路角色：**

C++

```
ENetRole LocalRole = GetLocalRole();
```

- **取得遠端網路角色：**

C++

```
ENetRole RemoteRole = GetRemoteRole();
```

5. 引擎設定與除錯技巧 (Blueprint & Debugging)

自訂藍圖函式庫 (Blueprint Function Library)： 講者示範了在 C++ 建立繼承自 `UBlueprintFunctionLibrary` 的靜態函式 `PrintLocalNetRole(AActor* Actor)`，並加上 `UFUNCTION(BlueprintCallable)` 巨集，方便在任何藍圖中隨時呼叫與印出 Log。

實際測試的關鍵觀察 (BeginPlay 陷阱)：

在 2 人連線的 Listen Server 中測試 Character 的 `BeginPlay` 時，會印出 **4 條 Log**（因為伺服器和客戶端各會載入 2 個角色）。

伺服器端觀察： 2 個角色的 Role 都是 `Authority`。

客戶端觀察： 沒有權威。其中 1 個是 AutonomousProxy（玩家自己），另 1 個是 SimulatedProxy（另一個玩家）。

UE5 物件附加與網路同步（Attachment & Replication）

1. 多人連線的「黃金法則」（The Golden Rule）

核心觀念： 任何重要的遊戲機制（如：穿戴裝備、造成傷害），都必須具有**權威性 (Authoritative)**，一律交由**伺服器 (Server)** 主導執行。

實作規則： 在觸發附加邏輯（如 Overlap 事件）時，務必先使用 if (HasAuthority()) 進行權限檢查，確保只有伺服器能執行附加指令。

2. Actor 層級 vs. Component（元件）層級附加

附加層級	適用情境	使用函式	網路同步成功之「必備條件」
Actor 附加	將整個裝備 Actor 附加到玩家 Actor 上（不指定精確的骨骼位置）。	AttachToActor()	Actor 本身必須開啟同步 (bReplicates = true)。伺服器的附加行為會自動向下同步。
Component 附加	將裝備精準附加到角色模型上的特定部位（例如：名為 s_head 的骨骼插槽 Socket）。	AttachToComponent()	**致命陷阱**：除了 Actor 要同步，**進行附加的「元件」本身也必須開啟同步！**

3. C++ 實作範例（裝備附加與元件同步）

以下示範如何將一個裝備網格體（SphereMesh），精準且同步地附加到角色的頭部插槽（Socket）：

```
// 1. 建構子 (Constructor) 設定
AMP_Armor::AMP_Armor()
{
    // 確保 Actor 本身會進行網路同步
    bReplicates = true;

    SphereMesh = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("SphereMesh"));
    SphereMesh->SetupAttachment(RootComponent);

    // 🚨 關鍵：若要讓 AttachToComponent 能在網路上同步，元件本身必須設為 Replicated！
    // 注意：修改此項無法使用 Live Coding，必須關閉編輯器重新編譯。
    SphereMesh->SetIsReplicated(true);
}
```

```
// 2. 覆寫重疊事件 (Overlap Event)
void AMP_Armor::NotifyActorBeginOverlap(AActor* OtherActor)
{
    Super::NotifyActorBeginOverlap(OtherActor);

    // 🏆 黃金法則：確保只有伺服器 (Server) 能執行附加邏輯
    if (HasAuthority())
    {
        ACharacter* PlayerChar = Cast<ACharacter>(OtherActor);
        if (PlayerChar)
        {
            // 將網格體附加到角色的骨架網格體上，並指定 Socket 名稱為 "s_head"
            SphereMesh->AttachToComponent(
                PlayerChar->GetMesh(),
                FAttachmentTransformRules::SnapToTargetNotIncludingScale,
                FName("s_head")
            );
        }
    }
}
```

4. 效能與架構考量 (Tips & Warnings)

頻寬負擔 (Overhead)： 雖然在建構子中使用 `SetIsReplicated(true)` 可以完美解決元件附加的同步問題，但如果角色身上有數十個裝備（頭盔、手套、左右鞋子...），**每個小元件都開啟同步會大幅增加網路頻寬的負擔**。

實務建議： 這堂課的作法是為了展示底層的附加機制原理。在大型多人連線專案中，通常會有更進階、更節省頻寬的設計模式（例如透過 RPC 或 RepNotify 變數來驅動客戶端自行生成純視覺用的裝備）。

UE5 變數同步與介面應用 (Variable Replication & Interfaces)

1. 降低耦合的利器：C++ 介面 (C++ Interfaces)

在多人連線遊戲中，裝備（Armor）不該綁死特定的角色類別（Character）。透過**介面 (Interface)** 可以讓程式碼更乾淨、擴展性更高。

宣告介面： 使用 `UFUNCTION(BlueprintCallable, BlueprintNativeEvent)` 將介面函式公開，方便在藍圖與 C++ 中雙向使用。

角色實作介面 (Character)：

公開繼承介面類別（例：`class AMyCharacter : public ACharacter, public IIMP_Player`）。

在 C++ 覆寫時，函式名稱後方必須加上 `_Implementation`（例：`GrantArmor_Implementation`）。

呼叫介面 (Armor)：

不需要強制轉型 `Cast<ACharacter>`!

先檢查是否實作：`OtherActor->Implements<UMP_Player>()`。

透過 `Execute_` 前綴呼叫：`IIMP_Player::Execute_GrantArmor(OtherActor, ArmorValue);`。

2. 變數同步的核心三部曲 (3 Steps for Variable Replication)

想讓伺服器上的變數變化（例如獲得護甲值）自動更新到所有客戶端畫面上，必須在 C++ 中嚴格執行以下三個步驟：

// 步驟一：在標頭檔 (.h) 中，為變數加上 Replicated 巨集標記

UPROPERTY(Replicated)

float Armor;

// 步驟二：在標頭檔 (.h) 中，宣告並覆寫 GetLifetimeReplicatedProps 函式

```
virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const
override;

// 步驟三：在實作檔 (.cpp) 中註冊變數

#include "Net/UnrealNetwork.h" //      必須引入這個標頭檔

void AYourCharacter::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps)
const

{

    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    // 使用 DOREPLIFETIME 巨集來註冊需要同步的變數

    DOREPLIFETIME(AYourCharacter, Armor);

}
```

3. 注意事項與避坑指南 (Tips & Warnings)

黃金法則再確認 (Authority Check)： 修改同步變數的值（例如 `Armor = ArmorAmount;`），這個動作**必須且只能在伺服器端執行**！如果客戶端自己修改了這個值，它不會同步給任何人，甚至可能會在下一次伺服器同步時被覆寫（強制拉回）。

最常見的新手失誤 (The Pre-requisite Trap)： 變數同步的大前提是：**「擁有該變數的 Actor 本身必須是可同步的」**。如果你做足了上述三部曲，但忘記將該 Actor 的 `bReplicates` 設為 `true`，變數同步絕對不會生效！

UE5 變數同步通知 (Rep Notifies)

1. 核心概念 (Core Concepts)

Rep Notify 的作用： 當伺服器端的變數發生改變，並將新值同步（Replicate）到客戶端時，系統會在客戶端自動觸發一個綁定好的特定函式。

命名慣例： 綁定的函式必須是無回傳值 (`void`)，且通常命名為 `OnRep_`變數名稱（例如變數叫 `Armor`，函式就叫 `OnRep_Armor`）。

2. C++ 語法與實作 (C++ Implementation)

要將變數與同步通知函式綁定，需要修改原有的 UPROPERTY 標記，並宣告對應的 UFUNCTION。

```
// 1. 在變數宣告時，使用 ReplicatedUsing 指定要觸發的函式名稱
UPROPERTY(ReplicatedUsing = OnRep_PickupCount)
int32 PickupCount;

// 2. 宣告對應的同步通知函式（必須加上 UFUNCTION 讓反射系統抓得到）
UFUNCTION()
void OnRep_PickupCount();

// （實作檔 .cpp 中）
void AYourCharacter::OnRep_PickupCount()
{
    // 當 PickupCount 變數同步到客戶端時，這裡的邏輯會自動執行
    // 例如：更新畫面上的 UI、播放撿取音效等
}
```

3. 進階技巧：獲取同步前的「舊值」 (Previous Value Trick)

這是一個非常強大且實用的隱藏技巧！如果你想知道變數在更新之前的值是多少（例如用來計算血量扣了多少），只需在 OnRep_ 函式中加入一個**與該變數型別完全相同**的參數即可。

```
// 在函式簽名中加入同型別的參數 (例如 int32)
UFUNCTION()
void OnRep_PickupCount(int32 PreviousValue);

void AYourCharacter::OnRep_PickupCount(int32 PreviousValue)
{
    // PreviousValue 會自動帶入「同步發生前」的舊數值
    // PickupCount 則是已經同步過來的「最新數值」
}
```

4. 注意事項與避坑指南 (Tips & Warnings)

C++ 與藍圖的行為差異 (The Server Call Trap)： 這是開發者從藍圖轉戰 C++ 時最常遇到的陷阱！

開發環境	伺服器端修改變數時，會自動觸發 Rep Notify 嗎？	解決方案
Blueprint (藍圖)	**會。** 伺服器修改變數的瞬間，預設連伺服器端都會觸發 Notify 節點。	無需額外處理。
C++ (本堂課)	**不會。** Rep Notifies 預設只會在接收同步的客戶端上自動觸發。	若伺服器也需要執行該邏輯，必須在修改變數後，手動呼叫該函式（例如 PickupCount++; OnRep_PickupCount();）。

重點結論： 如果你在伺服器上（例如撿起物品時）增加了數量，卻發現伺服器端自己的畫面上沒有印出 Debug 文字或更新 UI，這不是 Bug，而是因為 C++ 不會自動幫伺服器呼叫 Rep Notify！

UE5 變數同步條件 (Replication Conditions)

1. 核心概念 (Core Concepts)

為什麼需要同步條件？ 預設的變數同步 (DOREPLIFETIME) 會將變數的改變廣播給「所有」連線的客戶端。若變數只與單一玩家有關（例如：玩家自己背包裡的金幣、當前彈匣的彈藥量），廣播給所有人會造成極大的網路頻寬浪費。

解決方案： 透過加入條件列舉值（Enum），我們可以精確控制變數**「只在什麼時候」或「只同步給誰」**。

2. C++ 語法與實作 (C++ Implementation)

在 `GetLifetimeReplicatedProps` 函式中，不再使用預設的巨集，而是改用帶有條件的 **`DOREPLIFETIME_CONDITION`** 巨集。

```
void
AYourCharacter::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>&
OutLifetimeProps) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    // 參數 1: 類別名稱
    // 參數 2: 變數名稱
    // 參數 3: 同步條件 (從 CoreNetTypes.h 中選擇)
    DOREPLIFETIME_CONDITION(AYourCharacter, Armor, COND_OwnerOnly);
}
```

3. 常見同步條件一覽表 (Common Replication Conditions)

同步條件 (Enum)	觸發時機與同步對象	適用情境舉例
COND_InitialOnly	僅在 Actor 剛載入/生成的「初始封包」同步一次，後續數值改變**不會**再同步。	角色初始外觀設定、玩家初始名稱等不變屬性。
COND_OwnerOnly	僅同步給**擁有該** Actor (Owner) 的客戶端連線。	玩家私有資訊（如：個人彈藥量、金錢、背包物品）。
COND_SkipOwner	同步給除了擁有者以外的「所有其他客戶端」。	玩家本機已經處理好視覺預測，只需讓其他玩家看見的狀態。
COND_AutonomousOnly	僅同步給本地網路角色為 Autonomous Proxy 的客戶端。	類似 OwnerOnly，但基於網路角色 (Net Role) 判定。
COND_SimulatedOnly	僅同步給網路角色為 Simulated Proxy 的客戶端。	僅需其他玩家看到的特效或動畫變數。
COND_InitialOrOwner	初始生成時同步給所有人，但後續的數值改變**只同步給擁有者**。	所有人都要看見初始護甲值，但後續扣血只有玩家自己需要知道。

4. 注意事項與避坑指南 (Tips & Warnings)

所有權 (Ownership) vs. 角色 (Role)： 這是兩個不同的底層觀念。「所有權」是指你的連線 (Connection) 擁有 Player Controller，進而擁有 Character；而「角色 (Role)」是指 Actor 在特定機器上的身份（如 Autonomous/Simulated Proxy）。這決定了你要選擇 `OwnerOnly` 還是 `AutonomousOnly`。

進階預告 (COND_Custom)： 如果上述的靜態條件都無法滿足需求，UE5 還提供了一個 COND_Custom 條件，允許開發者透過程式邏輯「動態」開啟或關閉特定變數的同步.

UE5 動態同步控制 (Custom Replication Conditions)

1. 核心概念 (Core Concepts)

什麼是自訂同步？ 使用 COND_Custom 條件，允許開發者透過程式邏輯（通常是一個布林值），在遊戲執行期間（Runtime）隨時「開啟」或「關閉」特定變數的網路同步。

行為特徵 (Catch-up Effect)： * 當條件設為 false (關閉) 時，伺服器端的變數變化會被隱藏，不讓客戶端知道。

當條件瞬間切換為 true (開啟) 時，系統會立刻拿取伺服器端「最新、最權威」的數值向下同步，並觸發客戶端的 Rep Notify，瞬間修正客戶端的狀態。

2. C++ 實作四部曲 (Implementation Steps)

要達成動態同步控制，必須在 C++ 中嚴格執行以下四個步驟：

步驟一：宣告條件變數 在標頭檔 (.h) 中宣告一個布林值，作為控制同步的開關。

```
bool bReplicatePickupCount = false; // 預設關閉同步
```

步驟二：將同步條件設為 COND_Custom 在 GetLifetimeReplicatedProps 函式中，將變數的同步條件指定為自訂。

```
DOREPLIFETIME_CONDITION(AYourCharacter, PickupCount, COND_Custom);
```

步驟三：覆寫 PreReplication 函式並綁定條件 這是最關鍵的一步。在標頭檔宣告並在實作檔 (.cpp) 覆寫這個虛擬函式，將「要同步的變數」與「開關布林值」綁定。

```
// 標頭檔宣告
virtual void PreReplication(IRepChangedPropertyTracker&
ChangedPropertyTracker) override;

// 實作檔
void AYourCharacter::PreReplication(IRepChangedPropertyTracker&
ChangedPropertyTracker)
{
    Super::PreReplication(ChangedPropertyTracker);
```

```
// 參數：類別本身 (ThisClass), 變數名稱, 布林條件變數
DOREPLIFETIME_ACTIVE_OVERRIDE(ThisClass, PickupCount,
bReplicatePickupCount);
}
```

步驟四：修改 Build.cs（ 極度重要）
為了使用上述的巨集，必須打開專案的［專案名稱].Build.cs 檔案，在 PublicDependencyModuleNames 清單中手動加入 "NetCore" 模組。若忘記這步，編譯時會產生連結器錯誤 (Linker errors)。

3. 注意事項與避坑指南 (Tips & Warnings)

布林值開關只看「伺服器端」的臉色： 用來控制同步狀態的布林值（如 `bReplicatePickupCount`），**只有在伺服器端的值才有意義**。如果你在客戶端機器上把這個布林值改成 `true`，是毫無作用的，因為網路同步永遠是「由上往下（Server -> Client）」單向傳輸。

效能考量： 雖然動態開關非常強大，但頻繁且快速地切換同步狀態可能會增加引擎底層的運算與頻寬負擔，請謹慎使用。

預告： 既然客戶端無法直接改變伺服器的布林值，那我們該如何讓客戶端把資訊「向上」傳給伺服器呢？這就必須仰賴後續會學到的 **RPCs (Remote Procedure Calls, 遠端程序呼叫)** 機制！

UE5 擁有權與網路連線 (Ownership & Net Connection)

1. 核心概念：Actor 的擁有者 (Owner)

每個 Actor 都有一個 `Owner` 變數（型別同樣是 Actor），用來表示誰「擁有」它。

手動設定與取得： 可透過 C++ 的 `SetOwner()` 與 `GetOwner()` 來操作。

預設狀態： 直接放置在關卡中的 Actor，其 `Owner` 預設為空指標 (Null)。

自動綁定機制： 當一個 Pawn/Character 被玩家控制器 (Player Controller)「控制 (Possess)」時，該 Pawn 的 `Owner` 就會自動被設定為這個 Player Controller。

2. 核心概念：擁有網路連線 (Owning Net Connection)

在用戶端-伺服器架構中，連線歸屬決定了資料的流向。

連線建立： 當客戶端連線到伺服器時，伺服器會建立一個 `UNetConnection` 物件來代表這條網路連線，並將其與該玩家的 Player Controller 綁定。

階層追溯 (Ownership Hierarchy)： 引擎為了確認一個 Actor 屬於哪個客戶端，會不斷向上追溯它的 `Owner`。例如：裝備 Actor -> `Owner` 是 Pawn -> `Owner` 是 Player Controller -> 找到對應的 `UNetConnection`。

無連線狀態： 如果一個 Actor 沒有 Owner，或者向上追溯不到 Player Controller，它就不具備「擁有網路連線」。

3. 所有權的兩大實際應用 (Replication & Relevancy)

有了網路連線的歸屬後，引擎就能精準控制資料要傳給誰：

應用機制	作用與說明	核心屬性/設定
同步條件 (Rep Conditions)	決定同步變數「只傳給特定的客戶端機器」。	COND_OwnerOnly：系統會抓取 Actor 的「擁有網路連線」，並只將變數同步給該連線對應的機器（如玩家自己的護甲值）。
網路相關性 (Net Relevancy)	決定整個 Actor 的更新是否要同步給特定玩家（節省頻寬）。	bOnlyRelevantToOwner：僅對擁有者相關。 bAlwaysRelevant：永遠強制同步給所有人。 bNetUseOwnerRelevancy：完全沿用 Owner 的相關性設定。

4. 注意事項與避坑指南 (Tips & Warnings)

Player Controller 的「神隱」特性： `Player Controller` 類別預設開啟了 `bOnlyRelevantToOwner = true`。這代表在你的電腦（客戶端）上，**永遠只會存在你自己的 Player Controller**！你絕對看不到其他玩家的 Player Controller（只有伺服器端能看到所有人的）。

區分「權威」與「所有權」：

權威 (Authority)： 決定「誰有資格修改數值」（永遠是伺服器 Server）。

所有權 (Ownership)： 決定「修改後的資料要傳給哪台機器」（特定的客戶端 Client）。

Listen Server 的例外： 如果玩家是擔任傾聽伺服器 (Listen Server) 的房主，因為他本身就在伺服器機器上遊玩，所以該玩家不需要（也不會產生）`Net Connection`。

UE5 Actor 元件網路同步 (Actor Component Replication)

1. 核心概念 (Core Concepts)

擁有權 (Ownership)： Actor 元件的擁有者 (Owner) 會自動被設定為「建構/附加它的那個 Actor」。

同步機制的差異： 將「元件實體」設定為同步，與將「一般變數」設定為同步的方法完全不同。元件**不需要**（也不能）使用 `UPROPERTY(Replicated)` 標記。

2. 啟用元件同步的三種方法

要讓一個 Component 具備網路同步能力，你可以選擇以下三種方式之一來開啟：

從元件本身的建構子 (C++)： * *注意：在元件自己的建構子中，必須呼叫 `SetIsReplicatedByDefault(true)`，不可使用 `SetIsReplicated`，否則引擎會報錯。*

從擁有者 Actor 的建構子 (C++)： * 在使用 `CreateDefaultSubobject` 建構完元件後，呼叫 `Component 變數->SetIsReplicated(true)`。

從藍圖編輯器 (Blueprint)： * 在 Actor 藍圖的細節面板 (Details) 中，直接勾選 **Component Replicates**。

3. C++ 實作範本 (C++ Implementation)

方法一：在 Actor 端設定元件同步

```
// AYourCharacter 的建構子 AYourCharacter::AYourCharacter() { // 1. 建立元件 HealthComponent =
```

```
CreateDefaultSubobject<UMP_HealthComponent>(TEXT("HealthComponent"));
// 2. 開啟元件同步 HealthComponent->SetIsReplicated(true); }
```

方法二：在 Component 端預設開啟同步

```
// UMP_HealthComponent 的建構子
UMP_HealthComponent::UMP_HealthComponent() { // 必須使用 ByDefault 版本 SetIsReplicatedByDefault(true); }
```

4. 元件內的變數同步 (Replicating Variables inside Components)

只要元件本身已經開啟了同步 (Component Replicates = true)，你就可以在元件內部加入各種同步變數（例如：Health 生命值）。

> **實作規則完全相同：** 在元件內設定同步變數的流程，與在 Actor 內**一模一樣**（一樣需要標記 UPROPERTY(Replicated)、覆寫 GetLifetimeReplicatedProps 函式，並呼叫 DOREPLIFETIME 巨集）。

5. 注意事項與避坑指南 (Tips & Warnings)

> **最大新手陷阱 (The Prerequisite Trap)：** 如果你在 Component 裡面辛辛苦苦寫好了變數同步的三部曲，甚至寫了 OnRep_ 通知，但在測試時卻發現變數完全沒有同步過去，請第一時間檢查：**這個元件本身的 Component Replicates 開啟了嗎？**
>
> > **黃金法則：** 如果元件本身沒有開啟同步，它裡面包含的「所有同步變數」都不會進行網路同步！

UE5 遠端程序呼叫：Client RPC (Remote Procedure Calls)

1. 核心概念 (Core Concepts)

> **什麼是 RPC？** RPC 允許你在某一台機器（例如伺服器）上呼叫函式，但實際執行的地點卻在另一台特定的機器（例如客戶端）上。
>
> **RPC 的三大類型：**

Client RPC： 伺服器呼叫　特定客戶端執行。（本堂課重點）

Server RPC： 客戶端呼叫　伺服器執行。

NetMulticast (多播)： 伺服器呼叫　所有人（伺服器與所有客戶端）執行。

Client RPC 運作先決條件： 該 Actor 必須擁有**「網路連線 (Owning Net Connection)」**，系統才知道要把執行指令封包送給哪一台機器。

2. C++ 語法與實作 (C++ Implementation)

實作一個 Client RPC 必須嚴格遵守以下三個步驟與命名慣例：

步驟一：在標頭檔 (.h) 宣告巨集與函式

強烈建議加上 `Client_` 前綴以利辨識。

巨集參數 `Reliable` 代表必定送達（保證執行），若允許網路擁塞時遺失則可改用 `Unreliable`。

```
UFUNCTION(Client, Reliable)

void Client_PrintMessage(const FString& Message);
```

步驟二：在實作檔 (.cpp) 加上 `_Implementation` 後綴

雖然宣告時沒有後綴，但在實作時**必須**加上 `_Implementation`，這是因為 UE 的反射系統會在背景自動生成網路傳輸程式碼。

```
void AYourCharacter::Client_PrintMessage_Implementation(const FString& Message) { // 這裡的邏輯會在指定的客戶端機器上執行 // 例如：播放專屬音效、顯示 UI 提示等 }
```

步驟三：在伺服器端呼叫

呼叫時**不需要**加上後綴，直接使用原名稱即可。務必確保是由伺服器發起呼叫。

```
if (HasAuthority()) { Client_PrintMessage(TEXT("這段訊息將傳送到客戶端執行")); }
```

3. 注意事項與避坑指南 (Tips & Warnings)

BeginPlay 的時間差陷阱： 在 `BeginPlay` 發生的當下，角色的網路連線可能尚未建立完成。如果 Client RPC 找不到網路連線，它會**直接在呼叫它的機器（伺服器本地）**

上執行。實務上通常需要加入 Timer 延遲呼叫，或在確保連線建立後的生命週期再呼叫。

呼叫時機	狀態分析	執行結果
BeginPlay	角色通常尚未被 Controller 附身，處於無 Owner、無網路連線的狀態。	✕ **失敗**（會退回伺服器端本地執行）
PossessedBy	伺服器確認角色已綁定玩家控制器，正式取得對應客戶端的網路連線。	✓ **成功**（精準傳送到客戶端執行）
Timer **延遲呼叫**	強制等待數秒，確保連線與所有權的建置流程徹底跑完。	✓ **成功**（保險做法，但不夠即時）

Listen Server (房主) 的例外狀態： 擔任主機的玩家因為本身就在伺服器端遊玩，**沒有外部網路連線**。因此，針對房主角色呼叫的 Client RPC，永遠只會在伺服器端（本地）執行。

動態生成 Actor 的所有權 (Spawn Ownership)： 如果你要在一般的 Actor（非 Character）身上呼叫 Client RPC，該 Actor 必須被賦予網路連線。在伺服器使用 `SpawnActor` 生成物件時，**必須透過 `FActorSpawnParameters` 明確設定 `Owner`**（例如指向 Character），該 Actor 才能繼承網路連線，成功將 RPC 送達客戶端。

4. Client RPC 執行對照表 (Execution Matrix)

如果你對 Client RPC 到底會在哪裡執行感到疑惑，請參考以下由伺服器端發起呼叫時的底層規則：

該 Actor 的擁有連線 (Owning Connection)	實際執行地點 (Execution Location)	說明
Client (特定客戶端)	對應的 Client	正常運作：精準傳送到負責控制的玩家畫面上執行。
Server (伺服器本身)	Server	例如 Listen Server 的房主角色，因無外部連線，留在原地執行。
None (無連線)	Server	例如未設定 Owner 的野生 Actor 或過早呼叫，找不到目標，留在原地執行。

UE5 遠端程序呼叫：Server RPC (Remote Procedure Calls)

1. 核心概念 (Core Concepts)

什麼是 Server RPC？ 允許在**客戶端 (Client)** 發起呼叫，但實際的程式碼邏輯會在**伺服器 (Server)** 上執行。這是玩家向伺服器發送輸入請求的唯一正規途徑。

執行大前提： 當客戶端想要呼叫 Server RPC 時，該 Actor **必須具備屬於該客戶端的「擁有網路連線 (Owning Net Connection)」**，否則請求會被引擎直接判定為無效並丟棄 (Dropped)。

2. C++ 語法與實作 (C++ Implementation)

實作 Server RPC 的步驟與 Client RPC 非常相似，只是巨集標記不同：

步驟一：在標頭檔 (.h) 宣告巨集與函式

建議加上 `Server_` 前綴以利辨識。

使用 `Server` 巨集，並搭配 `Reliable` (保證送達) 或 `Unreliable` (允許網路擁塞時遺失)。

```
UFUNCTION(Server, Reliable)

void Server_PrintMessage(const FString& Message);
```

步驟二：在實作檔 (.cpp) 加上 `_Implementation` 後綴

真正的執行邏輯寫在這裡，這段程式碼將在**伺服器端**運行。

```
void AYourCharacter::Server_PrintMessage_Implementation(const FString& Message) { // 這裡的邏輯只會在 Server 上執行 // 例如：扣除玩家金幣、驗證傷害、修改同步變數等 }
```

步驟三：從客戶端發起呼叫

直接呼叫原函式名稱。通常會綁定在玩家的輸入事件（如按下鍵盤 E 鍵）中。

```
// 綁定在玩家輸入的函式中 Server_PrintMessage(TEXT("Please run this on the server."));
```

3. 注意事項與避坑指南 (Tips & Warnings)

孤兒 Actor 的沉默 (The Nullptr Owner Trap)： 如果你動態生成 (Spawn) 了一個 Actor，但**沒有設定它的 Owner**，這個 Actor 就不會有網路連線。此時如果客戶端試圖呼叫這個 Actor 的 Server RPC，封包會直接被引擎丟棄，什麼事都不會發生！

實用的生命週期檢查 `OnRep_Owner`： `Owner` 屬性本身是會網路同步的。當伺服器設定好 Actor 的 Owner 並將其同步到客戶端時，會自動觸發 `OnRep_Owner` 虛擬函式。如果你想在客戶端確認 Actor 已經拿到權限並準備好發送 RPC，覆寫並在這個函式裡呼叫 Server RPC 是非常安全的做法。

4. Server RPC 執行對照表 (Execution Matrix)

根據官方文件，Server RPC 的執行地點取決於「是誰呼叫的」以及「Actor 屬於誰」：

呼叫的機器 (Calling Machine)	該 Actor 的擁有連線 (Owning Connection)	實際執行地點 (Execution Location)	說明
Client (客戶端)	與發出呼叫的 Client 相符	Server	✅ **正常運作：**玩家操控自己的角色發送請求給伺服器。
Client (客戶端)	不相符 或 None (無連線)	無 (封包被丟棄 Dropped)	❌ **請求無效：**你不能控制別人的角色，也不能控制沒有 Owner 的物件發送請求。
Server (伺服器本身)	任何狀態	Server	伺服器呼叫自己的 Server RPC，會直接在本地端執行。

UE5 遠端程序呼叫：Multicast RPC (多播)

1. 核心概念 (Core Concepts)

什麼是 Multicast RPC？ 從**伺服器 (Server)** 發起呼叫，並在**伺服器端本身以及所有客戶端 (Clients)** 上同時執行的函式。

判定條件（與 Client/Server RPC 的最大差異）： * 前兩者依賴「擁有網路連線 (Owning Net Connection)」。

Multicast RPC **完全不看擁有權**，而是依賴**「網路相關性 (Net Relevancy)」**。只要這個 Actor 對該客戶端具有相關性（例如在玩家的視野範圍內），該客戶端就會收到並執行這個 RPC。

2. C++ 語法與實作 (C++ Implementation)

實作步驟與另外兩種 RPC 幾乎一樣，主要差異在於使用的巨集：

步驟一：在標頭檔 (.h) 宣告巨集與函式

建議加上 `Multicast_` 前綴以利辨識。

巨集參數必須使用 `NetMulticast`。

```
UFUNCTION(NetMulticast, Reliable) void Multicast_PrintMessage(const FString& Message);
```

步驟二：在實作檔 (.cpp) 加上 _Implementation 後綴

```
void AYourCharacter::Multicast_PrintMessage_Implementation(const FString& Message) { // 這裡的邏輯會在 Server 以及所有具相關性的 Client 上執行 // 適用情境：播放全域爆炸音效、觸發王關進場特效、顯示全服公告等 }
```

3. 注意事項與避坑指南 (Tips & Warnings)

絕對不要從客戶端呼叫： Multicast RPC 的設計初衷就是「由伺服器統一下達廣播」。如果你在客戶端機器上呼叫了 Multicast RPC，它**只會在那一台客戶端上本地執行**，其他人都收不到，這在多人連線中是毫無意義的！

重複執行的錯覺 (Log 數量加倍)： 在編輯器測試時（尤其是模擬 Dedicated Server 模式），如果你場景上有兩個角色同時觸發了 Multicast，你的 Log 會出現兩倍的數量。因為「每個角色」都在伺服器上發出了一次廣播，而「每個客戶端」都同時接收到了這兩次廣播。這不是 Bug，而是邏輯正常運作的證明。

Remote RPC 的存在： 引擎中還有一個相對罕見的 `Remote` RPC 設定，其呼叫方式與其他 RPC 大同小異。若未來開發有特殊需求，可以直接查閱官方文件的對照表。

4. Multicast RPC 執行對照表 (Execution Matrix)

根據官方文件，Multicast 的執行規則非常單純且霸道：

呼叫的機器 (Calling Machine)	實際執行地點 (Execution Location)	說明
Server (伺服器)	Server + 所有具相關性的 Client	✅ **正確用法：** 伺服器廣播，所有人一起執行（例如爆炸特效）。
Client (客戶端)	僅限該發出呼叫的 Client	❌ **錯誤用法：** 毫無意義的本地自嗨，不要這麼做。

UE5 RPC 資料驗證與防作弊 (RPC Validation)

1. 核心概念 (Core Concepts)

防作弊 (Anti-Cheat)： 惡意玩家可能會使用外掛程式竄改本機記憶體（例如把血量改為 10,000），然後將這個不合理的數值透過 RPC 傳送給伺服器。

資料驗證 (Validation)： 為了防堵作弊，我們可以在伺服器執行 RPC 邏輯之前，先架設一道「檢查哨」，確認接收到的資料是否合理合法。

強制踢出 (Kick)： 如果驗證機制判定資料異常（回傳 `false`），伺服器會毫不留情地將發送該請求的網路連線切斷，直接把作弊玩家踢出遊戲。

2. C++ 語法與實作 (C++ Implementation)

要為 RPC 加上驗證功能，只需在原本的實作上擴充兩個步驟：

步驟一：在巨集中加入 `WithValidation` 標記 在標頭檔 (.h) 的 RPC 宣告中，加上 `WithValidation` 參數。這通常用於 `Server` RPC，但其實任何類型的 RPC 都能使用。

```
UFUNCTION(Server, Reliable, WithValidation) void
Server_PrintMessage(const FString& Message);
```

步驟二：實作 `_Validate` 驗證函式 不需要在標頭檔額外宣告！直接在實作檔 (.cpp) 中寫出帶有 `_Validate` 後綴的函式。這個函式必須回傳 bool 值（true = 通過；false = 判定作弊並踢人）。

```
bool AMyCharacter::Server_PrintMessage_Validate(const FString& Message)
{// 檢查收到的字串是否為空。若為空字串，則回傳 false 判定失敗。 // 這是一句簡潔老練的寫法，等同於 if(Message.IsEmpty()) return false; else return true; return !Message.IsEmpty(); }
```

3. 注意事項與避坑指南 (Tips & Warnings)

驗證條件的絕對自由： `_Validate` 函式不只能用來檢查傳入的參數（例如字串是否為空、數值是否在合理範圍內），你還可以在這裡檢查**「任何」**伺服器端的狀態條件（例如：玩家現在的狀態允許開槍嗎？他的冷卻時間到了嗎？）。

踢人的實際後果： 在實際測試中，一旦驗證函式回傳 `false`，該客戶端會立刻與伺服器斷線，並被強制送回專案預設的起始地圖 (Default starting map)。這是一個內建且極度有效的防禦手段！

UE5 效能最佳化：網路相關性 (Net Relevancy)

1. 核心概念 (Core Concepts)

為什麼需要相關性？ 玩家不需要知道離他十萬八千里外、甚至看不見的物件狀態。伺服器為了節省頻寬，**只會針對「對該客戶端具有相關性」的 Actor 進行網路更新**。

底層判定函式： `IsNetRelevantFor`。引擎會對每個 Actor 執行這個虛擬函式，來判定它對特定連線是否具有相關性。

2. 相關性判定規則 (IsNetRelevantFor 執行順序)

引擎會依照以下「優先順序」進行嚴格的條件篩選：

強制相關或主動參與 (最高優先級)：

如果 Actor 的 `bAlwaysRelevant = true`

或它是被 Pawn / Player Controller 擁有的

或它本身就是 Pawn

或該 Pawn 是動作（如發出噪音/傷害）的發起者 (Instigator)

判定：具備相關性。

繼承擁有者：

如果 `bNetUseOwnerRelevancy = true` 且有 Owner。

判定：完全沿用 Owner 的相關性。

僅限擁有者：

如果 `bOnlyRelevantToOwner = true`（且沒通過第 1 點的檢查）。

判定：不具相關性（對其他玩家隱形）。

骨架附加物：

如果 Actor 附加在另一個物件的骨架上。

判定：由它附著的基底 (Base) 決定。

隱藏且無碰撞：

如果 `bHidden = true` 且根元件 (Root Component) 沒有碰撞。

判定：不具相關性。

缺乏根元件 (警告)：

若無根元件，引擎會在 Log 跳出警告，要求你手動設為「永遠相關」(Always Relevant)。

基於距離的相關性 (預設最終防線)：

如果上述皆未攔截，引擎預設會檢查 Actor 與觀察者的距離。若距離小於 **`Net Cull Distance`** **(網路剔除距離)**。

判定：具備相關性。

UE5 頻寬分配：網路優先權 (Net Priority)

1. 核心概念 (Core Concepts)

什麼是優先權？ 當有大量 Actor 同時具備「相關性」且需要更新時，伺服器的頻寬不夠一次傳完。引擎會利用**優先權 (Priority)** 進行「負載平衡 (Load Balancing)」，決定誰先更新、誰獲得較多的頻寬。

數值意義 (浮點數 Float)： 預設值為 `1.0`。這是一個**「比例 (Ratio)」**概念。

舉例：優先權 `2.0` 的 Actor，獲得的網路更新頻率將會是優先權 `1.0` 的 Actor 的 **2** 倍。

2. 底層計算機制 (GetNetPriority)

在編輯器面板設定的 1.0 或 2.0 只是「基礎優先權」。引擎實際排隊時，是透過 GetNetPriority 函式動態計算的：

動態權重補償 (防止餓死)： 基礎優先權會乘上 **「距離上次同步後所經過的時間」**。這確保了即使是優先權極低的背景 Actor，只要等得夠久，它的動態權重也會逐漸攀升，最終獲得一次更新的機會。

空間權重： 函式也會將 Actor 與玩家的相對距離、視角位置納入計算考量。

3. 注意事項與避坑指南 (Tips & Warnings)

最常見的新手誤區：數值通膨 絕對不要因為覺得「所有物件都很重要」，就把所有 Actor 的優先權都改成 100.0！ 因為優先權看的是「比例」。如果大家都是 100.0，那比例依然是 1:1，跟全部都是 1.0 的效能表現完全一模一樣。 **正確做法：** 基準維持 1.0，只將最重要的核心 Actor（如玩家本身、高速飛行的子彈）調高至 2.0 或 3.0，將無關緊要的場景互動物件調低至 0.5。

UE5 遊戲模式與規則 (Game Mode)

1. 核心概念 (Core Concepts)

遊戲規則的制定者： Game Mode 負責定義整場遊戲的玩法、勝負條件、團隊設定等。

預設類別分配器：決定玩家加入時要生成哪些預設物件，包含 Default Pawn (預設棋子)、Player Controller (玩家控制器)、Game State、Player State 與 HUD。

玩家進出管理： 透過覆寫 PostLogin (登入後) 與 Logout (登出) 函式，可攔截並處理玩家加入與離開遊戲的邏輯。

2. 兩種內建模式比較 (GameModeBase vs. GameMode)

類別類型	特色與功能	適用場景
GameModeBase	提供最基礎的遊戲架構（設定預設類別、處理玩家進出）。	簡單的休閒遊戲、合作過關、不需要「比賽」概念的遊戲。
GameMode	繼承自基礎版。額外支援 Match State (比賽狀態) 與 Delayed Start (延遲開始) 機制。	競技型射擊遊戲、大亂鬥、需要「等待玩家準備」再開局的遊戲。

3. 延遲開始機制與 C++ 實作 (Delayed Start Implementation)

如果開啟了延遲開始，玩家剛加入時會處於「無實體」狀態（操作預設的幽靈 Spectator Pawn），直到比賽正式宣告開始，才會生成真正的玩家角色。

步驟一：在建構子中開啟延遲

AMP_GameMode::AMP_GameMode() { // 啟用延遲開始 bDelayedStart = true; }

步驟二：使用計時器手動啟動比賽

void AMP_GameMode::BeginPlay() { Super::BeginPlay(); // 設定計時器，5 秒後呼叫自訂的 StartDelayFinished 函式
GetWorldTimerManager().SetTimer(MatchStartTimer, this, &AMP_GameMode::StartDelayFinished, 5.0f, false); }

void AMP_GameMode::StartDelayFinished() { // 手動觸發比賽開始
StartMatch(); }

步驟三：覆寫 StartMatch

void AMP_GameMode::StartMatch() { // 必須呼叫 Super 版本，引擎才會開始生成真實的 Player Pawn Super::StartMatch();
GEngine->AddOnScreenDebugMessage(-1, 5.f, FColor::Orange, TEXT("Match has started!")); }

4. 引擎與藍圖設定 (Engine & Blueprint Setup)

替換預設類別： 建立基於 C++ Game Mode 的藍圖（例如 BP_GameMode），在細節面板中，將 **Default Pawn Class** 與 **Player Controller Class** 替換為你的自訂藍圖。

關卡生效： 打開關卡的 **World Settings (世界設定)** 面板，將 GameMode Override 設為你剛建立的 BP_GameMode。

5. 注意事項與避坑指南 (Tips & Warnings)

絕對的伺服器專屬 (Server-Only)：

Game Mode **只存在於伺服器上**，客戶端機器上完全沒有這個物件。

禁止同步： 絕對不要在 Game Mode 裡面放置任何同步變數 (`Replicated`) 或呼叫 RPC，它們不會有任何作用。

存取限制： 試圖在客戶端獲取 Game Mode，藍圖會回傳 `None`，C++ 會回傳 `nullptr`。在 Game Mode 裡檢查 `HasAuthority()` 也是多此一舉，因為它永遠都在伺服器上。

延遲開始的「輸入失效」陷阱 (Bonus 解答)：

在啟用 Delayed Start 後，若發現 5 秒後角色成功生成，但卻「無法移動/沒有輸入反應」，通常是因為**忘記在 Game Mode 中替換自訂的 Player Controller**。

UE5 的 Enhanced Input (輸入映射內容 IMC) 通常是在自訂的 Player Controller 中載入的。如果 Game Mode 生成了正確的角色，卻分配了預設的空 Controller，輸入系統自然會徹底失效。

UE5 遊戲狀態同步 (Game State)

1. 核心概念 (Core Concepts)

全域資料的公佈欄： Game State 負責儲存整場遊戲的狀態，例如隊伍分數、任務進度、玩家分隊名單等。

具備網路同步能力： 與伺服器專屬的 Game Mode 不同，Game State 是一個會進行網路同步 (Replicated) 的 Actor。伺服器上修改的同步變數，所有連線的客戶端都能接收到更新。

網路更新頻率設定：Game State 預設的網路更新頻率 (Net Update Frequency) 為 10（每秒更新 10 次），網路優先權 (Net Priority) 預設為 1。這比會快速移動的 Pawn 稍微慢一些，適合用來同步不需要極度即時的狀態資料。

Base 版本的對應鐵則： UE5 有嚴格的配對規則。如果你使用的是 `AGameMode`，就必須搭配 `AGameState`；如果是 `AGameModeBase`，就必須搭配 `AGameStateBase`。絕對不能混搭！

2. C++ 實作範本：簡易分隊系統 (Team System)

以下是透過 Game State 與 Game Mode 合作，在玩家登入時進行分隊並同步給所有人的標準流程：

步驟一：在 Game State 建立同步陣列

步驟二：在 Game Mode 攔截玩家登入並分隊

步驟三：在客戶端（例如 Character）查詢 Game State

3. 引擎與藍圖設定（Blueprint Setup）

替換預設 Game State： 建立基於 C++ Game State 的藍圖（例如 BP_GameState）。打開你之前建立的 BP_GameMode，在細節面板找到 **Game State Class**，將其替換為你自訂的 BP_GameState。

4. 注意事項與避坑指南（Tips & Warnings）

無所不在的存取權： Game State 是少數在伺服器端可以自由修改、在客戶端也能隨時透過 UGameplayStatics::GetGameState() 輕易取得並讀取的全域物件。

Player State 陣列： Game State 底層內建了一個儲存 Player State 的陣列。請注意，這個陣列本身不是同步變數，但陣列裡面存放的 Player State 物件本身是會進行網路同步的（這部分後續課程會深入探討）。

UE5 玩家狀態同步（Player State）

1. 核心概念（Core Concepts）

個別玩家的資料庫： Player State 專門用來儲存與「特定玩家」相關的資訊，例如：得分 (Score)、Ping 值、玩家名稱、所屬隊伍等。

具備網路同步能力 (Replicated)： Player State 會同步給**所有**連線的客戶端。

擁有網路連線 (Owning Net Connection)： 它與 Player Controller 共用同一個連線通道。這代表 Player State **具備發送 RPC 的能力**。

預設屬性：

> **Net Update Frequency (更新頻率)：** 預設為 **1**（每秒僅更新一次）。在需要即時反應的遊戲中，通常需要調高。
>
> **Net Priority (優先權)：** 預設為 1。

2. Player State vs. Player Controller（關鍵差異）

這是開發者最常混淆的地方，請務必記住：

特性	Player Controller (玩家控制器)	Player State (玩家狀態)
可見性	**僅伺服器與擁有者可見。** 你看不到別人的 Controller。	**所有人皆可見。** 你可以看到畫面上所有人的 Player State。
用途	處理輸入邏輯、控制 Pawn、發送私人 RPC。	儲存需要讓大家看到的個人統計數據（如排行榜、血量條）。

3. C++ 實作範本 (C++ Implementation)

步驟一：設定同步變數與 Getter/Setter

步驟二：在 Actor 或拾取物中修改數據 (伺服器端執行)

步驟三：在 Character 中讀取數據

4. 性能最佳化：網路更新頻率 (Net Update Frequency)

現象： 當 Net Update Frequency 設為 1 時，玩家撿起物品後立刻按下查詢鍵，可能會印出「舊的」數值。這是因為同步包還在排隊，最慢要等 1 秒才會送達。

解決方案：

藍圖： 在 BP_PlayerState 的細節面板搜尋 Net Update Frequency 並調高（如 10 或 100）。

C++： 在建構子中呼叫 SetNetUpdateFrequency(10.0f);。

5. 注意事項與避坑指南 (Tips & Warnings)

拾取邏輯的順序：在實作 NotifyActorBeginOverlap 時，務必先完成數值修改（如 SetNumPickups），最後才呼叫 Destroy()。如果先銷毀了物件，後續的程式碼可能無法正確執行。

所有人的狀態： 因為 Player State 是同步給所有人的，你可以輕易地透過 GameState->PlayerArray 遍歷所有玩家的 Player State 來製作遊戲排行榜 (Scoreboard)。

UE5 玩家控制器 (Player Controller)

1. 核心概念 (Core Concepts)

玩家的靈魂與意志： 從引擎的角度來看，Player Controller 代表了「玩家本人」。它負責控制 (Possess) 場上的 Pawn 或 Character（肉體）。

掌握網路連線 (Owning Net Connection)： 這是最重要的網路通訊樞紐。Player Controller 掌握著玩家的專屬網路連線，被它控制的 Pawn 以及附加的裝備，都是透過這條連線來決定所有權 (Ownership) 與發送 RPC。

輸入處理中心： 具備處理玩家輸入 (Input) 的能力，可透過覆寫 SetupInputComponent 將按鍵綁定到對應的函式上。

極高的網路權重： 為了確保玩家操作的極致流暢，它的預設網路更新頻率 (Net Update Frequency) 高達 **100**，網路優先權 (Net Priority) 高達 **3.0**。

2. 注意事項與避坑指南 (Tips & Warnings)

絕對隱私的網路相關性 (Only Relevant to Owner)： * Player Controller 只對它的擁有者具有相關性。

> **在客戶端上：** 你的電腦裡永遠**只會存在你自己的** Player Controller。如果你試圖獲取「其他玩家」的控制器，你只會得到一個空指標 (Null pointer)。
>
> **在伺服器上：** 只有伺服器（包含 Listen Server 的房主）能看到並擁有所有玩家的 Player Controller。

3. 架構設計指南：輸入 (Input) 該寫在哪裡？

在開發多人連線遊戲時，決定將輸入邏輯寫在 Character 還是 Player Controller 是一門架構哲學。請依循以下客觀標準來分類：

寫在 Character (肉體) 上	寫在 Player Controller (靈魂) 上
判斷原則： 該輸入是否強烈依賴於這個特定的物理實體？	**判斷原則：** 該輸入是否跨越了肉體的限制，屬於系統層級或玩家本身的意志？
適用情境： 移動 (WASD)、跳躍、衝刺、翻滾、攻擊、格擋。	**適用情境：** 開啟暫停選單 (ESC)、開啟地圖、切換背包、查看計分板、語音發話 (Push-to-talk)、UI 點擊互動。

寫在 Character (肉體) 上	寫在 Player Controller (靈魂) 上
架構優勢 (生死脫鉤)： 角色死亡或被摧毀時，輸入自然失效。不用寫一堆 if (IsDead) 來阻擋玩家在死後按空白鍵跳躍。玩家下車 (Unpossess) 後，也無法再按車子的喇叭。	**架構優勢 (靈魂不滅)：** 即使角色被秒殺、屍體被引擎回收 (Destroyed)，玩家依然能按下 ESC 退出遊戲或打開計分板。UI 和 HUD 通常也由它生成與管理。
網路連線考量： 適合處理需要讓所有玩家看見的動作（透過 Server RPC 傳遞後再廣播）。	**網路連線考量：** 極度私密安全。處理不需要讓別人知道的操作（例如滑鼠在背包介面裡點擊了哪個道具）。

UE5 玩家化身與載具 (Pawn & Character)

1. 核心概念 (Core Concepts)

玩家的化身 (Avatar)： `Pawn`（以及其繼承類別 `Character`）是玩家在遊戲世界中具體存在的實體載具。

網路連線的「租借」機制： `Pawn` 本身是沒有專屬網路連線的。只有當它被 `Player Controller` **控制 (Possess)** 的期間，它才會與 Controller 共享**「擁有網路連線 (Owning Net Connection)」**。一旦解除控制 (Unpossess)，它就會失去這條通訊管道。

全域可見性 (Replicated to All)： 為了讓玩家能看到彼此的動作，Pawn 預設會進行網路同步，廣播給所有連線的客戶端（前提是符合「網路相關性 Relevancy」的距離與條件判定）。

輸入處理能力： 具備 `SetupPlayerInputComponent` 函式，可以綁定與處理輸入動作（這部分可完全對應我們在上一堂課梳理出的「靈魂與肉體」輸入分類原則）。

2. 效能與同步規格 (Network Settings)

因為角色會頻繁移動、跳躍、觸發動畫，為了確保所有玩家看到的視覺與動作絕對流暢，引擎給予了它極高的同步特權：

Net Update Frequency (網路更新頻率)： 預設高達 **100**（每秒更新 100 次）。

Net Priority (網路優先權)： 預設高達 **3.0**。

(註：這兩項數值與極度重要的 `Player Controller` 完全一致。)

3. 架構設計指南：Character 該負責什麼？（Architecture Philosophy）

在多人連線的架構設計中，確保 Character 類別的乾淨與模組化是非常重要的：

設計面向	建議作法與思考方向
擴充能力 (Abilities)	**善用元件 (Components)：** 不要把生命值、耐力值、背包邏輯全塞進 Character 裡。應該將它們寫成獨立的 Actor Component 再附加到角色上。
狀態保存 (State)	**狀態抽離：** 把需要長久保存、不怕角色死亡消失的資料（如：擊殺分數、拾取總數、隊伍歸屬），通通交給上一堂課學過的 Player State 處理。
終極解決方案 (GAS)	**Gameplay Ability System：** 誠如筆記外的補充，Epic Games 官方對於「如何乾淨地賦予角色能力」給出的進階答案就是 GAS。未來若將技能、屬性、Buff/Debuff 全部封裝在 Ability System Component 中，能讓 Character 類別達到最完美的解耦。

UE5 介面與網路同步（HUD, Widgets & UI Replication）

1. 核心概念（Core Concepts）

裝飾性且絕對不同步： HUD 類別與 UMG (User Widget) 僅用於本地端畫面的呈現，它們**絕對不會**進行網路同步 (Replicate)。不要在上面做任何網路傳輸的邏輯。

本地擁有權 (Local Ownership)： UI 介面是非常私密的。Widget 應該只被建立、並關聯到「本地擁有 (Locally Owned)」的 Player Controller 上。

通訊橋樑 (Delegates/Event Dispatchers)： 既然 UI 不會同步，我們必須利用**動態多播委派 (Dynamic Multicast Delegates)**，讓 UI 去監聽會同步的類別（如 Player State）。當底層資料改變並觸發 Rep Notify 時，發出廣播讓 UI 更新畫面。

2. C++ 實作範本：透過 Delegate 更新 UI

這是一套非常標準且乾淨的前後端分離架構：

步驟一：在 Widget 中綁定藍圖 UI 元件

步驟二：在 Player State 宣告並廣播委派

步驟三：在 Player Controller 建立 UI 並綁定監聽

3. 注意事項與避坑指南（Tips & Warnings）

伺服器 Crash 陷阱 (IsLocalController)： 絕對不要在伺服器（特別是 Dedicated Server，因為它根本沒有畫面）上嘗試渲染或建立 UI！在建立任何 Widget 之前，務必加上 `if (!IsLocalController()) return;` 進行安全檢查。

綁定時機的陷阱 (OnRep_PlayerState vs BeginPlay)：

> **Client 端：** 在多人連線中，客戶端進入 `BeginPlay` 階段時，`Player State` 通常**還沒同步下來**（是 Null）。所以綁定 UI 監聽的最佳時機，是覆寫 Player Controller 的 `virtual void OnRep_PlayerState() override;` 函式。
>
> **Listen Server 端 (房主特例)：** 房主一開始就擁有 `Player State`，所以不會觸發 `OnRep_PlayerState`。因此實務上，房主需要在 `BeginPlay` 裡也寫一份綁定邏輯。

UI 數字慢半拍 (Net Update Frequency)： 如果你發現撿起物品後 UI 數字更新「卡卡的」，這是因為 `Player State` 預設的網路更新頻率只有 **1**（每秒更新一次）。請在 `Player State` 的建構子中呼叫 `SetNetUpdateFrequency(10.0f);`，讓 UI 的視覺回饋變得即時滑順。

UE5 多人連線關卡切換（Level Travel）

1. 核心概念：硬切換 vs. 無縫切換

在多人遊戲中轉換地圖時，主要有兩種處理方式：

比較項目	硬切換 (Hard Travel)	無縫切換 (Seamless Travel)
執行緒阻塞	阻塞呼叫 (Blocking)，暫停遊戲執行緒。	非阻塞呼叫 (Non-blocking)，背景非同步執行。
玩家連線狀態	客戶端會強制斷線，等待目標地圖載入後再重新連線。	客戶端保持連線狀態。
過渡處理	畫面卡頓 (Hitch)，直接進入漫長的載入畫面。	將玩家先移至極小的過渡地圖 (Transition Map) 暫存，背景同時載入目標地圖。
資料保留度	幾乎所有物件被銷毀（除了 Game Instance 等持久類別）。	玩家控制器 (Player Controller) 會被保留 (Persist)，非常適合存放跨關卡資料！

2. 切換類型與選項字串 (Travel Types & Options)

選項字串 (Options String)： 呼叫切換關卡時帶入的參數（以 ? 開頭並分隔，例如 ?listen）。它可以用來告訴新地圖要以什麼模式開啟。

三種切換類型 (Travel Types)： 決定了伺服器狀態與選項字串的去留。

切換類型 (Type)	運作行為	是否支援無縫切換	適用情境
相對 (Relative)	**不切斷連線**，保留選項字串。	✅ **支援**	在同一個伺服器內切換地圖（最常用）。
部分 (Partial)	伺服器重設，保留選項字串。	❌ 不支援 (硬切換)	需要重置伺服器狀態但保留設定時。
絕對 (Absolute)	伺服器重設，**清空選項字串**，使用絕對 IP。	❌ 不支援 (硬切換)	從主選單 (Standalone) 連線到外部的 Listen Server。

3. C++ 關卡切換核心函式

`UEngine::Browse`： 永遠執行**硬切換**，通常用於本地端或斷開所有連線重新開始。

`UWorld::ServerTravel`： **僅限伺服器呼叫。** 伺服器發起跳轉，並自動對所有連線的客戶端呼叫 `ClientTravel`，把所有玩家一起「拉」過去。

`APlayerController::ClientTravel`：

伺服器呼叫它： 指示特定的單一客戶端前往新地圖（保持當前伺服器連線）。

客戶端呼叫它： 客戶端主動切換並嘗試連線到一個「全新的伺服器 IP」。

4. 實作無縫切換：黃金 5 步驟 (Seamless Travel Setup)

要讓無縫切換成功運作，必須在引擎與程式碼中嚴格執行以下設定：

準備地圖： 準備好「過渡地圖 (Transition Map)」(建議全空或極小) 與「目標地圖」。

加入打包清單： 開啟專案設定 (Project Settings)，找到 **List of maps to include in a packaged build**，將過渡地圖與目標地圖都加進去。（最常忘記的致命錯誤！）

設定過渡地圖： 在專案設定的 `Maps and Modes` -> `Advanced` (進階) 中，指定你的 Transition Map。

啟用無縫切換設定： 在你要「離開」的那個 `Game Mode` 中，將 `bUseSeamlessTravel` 設為 `true`（可在 C++ 建構子或藍圖設定）。

執行切換邏輯： 在伺服器端（確保有 Authority）呼叫 `GetWorld()->ServerTravel("目標地圖名稱");`。

5. 注意事項與避坑指南 (Tips & Warnings)

絕對不要在 PIE (Play In Editor) 中測試關卡切換！

編輯器的預覽模式無法真實模擬網路切換底層，強行測試極易導致引擎崩潰 (Crash) 或產生錯誤的行為結果。

正確測試方法： 使用 **Standalone Game (獨立遊戲)** 模式啟動，或者將專案 **打包 (Package)** 後執行測試。